zpp.bits

C++26 module port of `zpp::bits`

A binary serialization and RPC library

User Manual

Module port based on `eyalz800/zpp_bits`.

Contents

1 Overview

`zpp.bits` is a C++26 named module providing binary serialization and a lightweight remote-procedure-call (RPC) layer. It is a port of the original single-header `zpp::bits` library to the C++ named-module system, with all pre-C++26 workarounds removed.

Compared to the original library:

- Distributed as a named module, not a header. There is no preprocessor surface, no include guards, no `#include`.
- C++26 is mandatory. The library relies on P1061 ("Structured bindings can introduce a pack") for automatic member detection, which removes the need for manual `using serialize = members<N>` declarations on most types.
- All deprecated paths from the header version — in particular `ZPP_BITS_AUTODETECT_MEMBERS_MODE` — are gone.
- The single TU has been split into a primary interface and ten partition interfaces. End users normally import only the primary; partitions are available for callers who want a finer-grained dependency surface.

The on-wire format is identical to the original library. Code written against the original header can typically be migrated by replacing `#include "zpp_bits.h"` with `import zpp.bits;` and removing redundant aggregate-count hints.

2 Module structure

The module is laid out as a primary interface unit plus ten partition interface units:

Unit	Contents
`zpp.bits` (primary)	Re-exports every partition. This is what you normally import.
`zpp.bits:core`	`kind`, `members`, `protocol`, `serialization_id`, `errc`, `access`, `destructor_guard`.
`zpp.bits:traits`	Internal type traits.
`zpp.bits:concepts`	Concepts used to dispatch serialization paths.
`zpp.bits:options`	`bytes`/`as_bytes`, the `options` inline namespace, `sized_item`/`sized_t`/`unsized_t`, `optional_ptr`.
`zpp.bits:varint`	`varint`, `varint_encoding`, the `vint*_t`/`vuint*_t`/`vsint*_t`/`vsize_t` aliases, `size_varint`.
`zpp.bits:archive`	`basic_out`, `out`, and `in` plus their deduction guides.
`zpp.bits:io`	`input`, `output`, `in_out`, `data_in_out`/`data_in`/`data_out`, `to_bytes`/`from_bytes`, `id`, `known_id`.
`zpp.bits:rpc`	`function_traits`, `value_or_errc`, `apply`, `bind`, `bind_opaque`, `rpc`.
`zpp.bits:protobuf`	`pb_protocol`, `pb_members`, `pb_field`, `pb_reserved`, `pb_map`, `pb_value`.
`zpp.bits:numbers`	`numbers::big_endian`, `sha1`/`sha256`, the `literals` inline namespace, string and vector size aliases.

Importing the primary module is the recommended path:

```
import zpp.bits;
```

If you only need a slice of the API and want to minimise BMI dependencies, you can import partitions directly from within the same module — though this is not the typical case.

3 Building

`zpp.bits` is a Clang-tested module targeting C++26. Other modern toolchains with C++26 module support should also work; the only language features the module depends on are P1061 (structured-binding packs) and `import std`.

A minimal Clang command for a quick smoke test:

```
# Build module BMIs in dependency order.
clang++ -std=c++26 --precompile zpp_bits-core.cppm     -o zpp.bits-core.pcm
clang++ -std=c++26 --precompile zpp_bits-traits.cppm   -fprebuilt-module-path=. -o
zpp.bits-traits.pcm
clang++ -std=c++26 --precompile zpp_bits-concepts.cppm -fprebuilt-module-path=. -o
zpp.bits-concepts.pcm
clang++ -std=c++26 --precompile zpp_bits-options.cppm  -fprebuilt-module-path=. -o
zpp.bits-options.pcm
clang++ -std=c++26 --precompile zpp_bits-varint.cppm   -fprebuilt-module-path=. -o
zpp.bits-varint.pcm
clang++ -std=c++26 --precompile zpp_bits-archive.cppm  -fprebuilt-module-path=. -o
zpp.bits-archive.pcm
clang++ -std=c++26 --precompile zpp_bits-io.cppm       -fprebuilt-module-path=. -o
zpp.bits-io.pcm
clang++ -std=c++26 --precompile zpp_bits-rpc.cppm      -fprebuilt-module-path=. -o
zpp.bits-rpc.pcm
clang++ -std=c++26 --precompile zpp_bits-protobuf.cppm -fprebuilt-module-path=. -o
zpp.bits-protobuf.pcm
clang++ -std=c++26 --precompile zpp_bits-numbers.cppm  -fprebuilt-module-path=. -o
zpp.bits-numbers.pcm
clang++ -std=c++26 --precompile zpp_bits.cppm          -fprebuilt-module-path=. -o
zpp.bits.pcm

# Compile user code.
clang++ -std=c++26 main.cpp -fprebuilt-module-path=. *.pcm -o main
```

Production builds should rely on a build system with C++ modules support (`clang-scan-deps`, P1689) rather than driving the compiler by hand.

3.1 Build-time configuration

A small number of inlining knobs from the original library survive in the port, but they are baked into the BMI at module-build time. They cannot be redefined per translation unit by importers — set them on the command line when precompiling the module, then live with the choice:

Macro	Effect
`ZPP_BITS_INLINE_DECODE_VARINT=1`	Force-inline the full varint decode path. Off by default to keep code size down.
`ZPP_BITS_INLINE_MODE=0`	Disable all forced inlining. Use if you suspect inlining is bloating your binary.

4 Introduction

For most types, serialization requires nothing more than the type declaration itself. Thanks to P1061, both aggregates and many non-aggregates work out of the box.

```
import zpp.bits;

struct person
{
    std::string name;
    int age{};
};
```

Serializing two persons through a byte vector:

```
// `data_in_out` returns a vector of bytes together with input and output
// archives, decomposable via structured bindings.
auto [data, in, out] = zpp::bits::data_in_out();

// Serialize a few people:
out(person{"Person1", 25}, person{"Person2", 35});

person p1, p2;

// Deserialize one by one (`in(p1); in(p2);`) or together:
in(p1, p2);
```

This compiles, but the compiler will warn about discarding the return values of `out()` and `in()`. Error handling is the next section.

5 Error Handling

The library offers three styles of error handling: return values, exceptions, and `zpp::throwing`-based coroutine error handling.

5.1 Using return values

The most explicit option, and the right choice if you prefer return values:

```
auto [data, in, out] = zpp::bits::data_in_out();

auto result = out(person{"Person1", 25}, person{"Person2", 35});
if (failure(result)) {
    // `result` is implicitly convertible to `std::errc`.
    // handle the error or return/throw exception.
}

person p1, p2;

result = in(p1, p2);
if (failure(result)) {
    // handle the error
}
```

5.2 Using exceptions

The exceptions-based way uses `.or_throw()` — read it as "succeed or throw":

```
import zpp.bits;
import std;

int main()
{
    try {
        auto [data, in, out] = zpp::bits::data_in_out();

        out(person{"Person1", 25}, person{"Person2", 35}).or_throw();

        person p1, p2;
        in(p1, p2).or_throw();

        return 0;
    } catch (const std::exception & error) {
        std::cout << "Failed with error: " << error.what() << '\n';
        return 1;
    } catch (...) {
        std::cout << "Unknown error\n";
        return 1;
    }
}
```

5.3 Using zpp::throwing

If `zpp::throwing` is available, error checking reduces to `co_await`:

```
int main()
{
    return zpp::try_catch([]() -> zpp::throwing<int> {
        auto [data, in, out] = zpp::bits::data_in_out();

        co_await out(person{"Person1", 25}, person{"Person2", 35});

        person p1, p2;
        co_await in(p1, p2);

        co_return 0;
    }, [](zpp::error error) {
        std::cout << "Failed with error: " << error.message() << '\n';
        return 1;
    }, [](/* catch all */) {
        std::cout << "Unknown error\n";
        return 1;
    });
}
```

`zpp::throwing` is an independent project; it is not part of the `zpp.bits` module.

6 Error Codes

All three error-handling styles share the same set of `std::errc` codes:

Code	Meaning
`result_out_of_range`	Attempting to write to or read from a too-short buffer.
`no_buffer_space`	Growing buffer would exceed allocation limits or overflow.
`value_too_large`	Varint encoding is beyond representation limits.
`message_size`	Message size exceeds user-defined allocation limits.
`not_supported`	Attempt to call an RPC that is not listed as supported.
`bad_message`	Attempt to read a variant of unrecognized type.
`invalid_argument`	Attempting to serialize a null pointer or a valueless variant.
`protocol_error`	Attempt to deserialize an invalid protocol message.

7 Serializing aggregates and non-aggregates

Under C++26, P1061 lets the library detect the number of members in both aggregates and many non-aggregates automatically. The plain declaration is usually all that is required:

```
namespace my_namespace
{
struct person
{
    person(auto && ...){/*...*/} // Non-aggregate constructor.

    std::string name;
    int age{};
};
} // namespace my_namespace
```

No `using serialize = members<N>;` declaration is needed. If for some reason you want to force a specific count anyway, the `members<N>` declaration is still honoured — it is mostly useful for types where P1061 cannot deduce the count.

8 Serializing private classes

If your data members or default constructor are private, befriend `zpp::bits::access`. Under C++26 no member count is required:

```
struct private_person
{
    friend zpp::bits::access;

private:
    std::string name;
    int age{};
};
```

9 Explicit Serialization

For full control — or for types where structured bindings are not viable — define an explicit `serialize` function. The shape is:

```
    constexpr static auto serialize(auto & archive, auto & self)
    {
        return archive(self.object_1, self.object_2, /* ... */);
    }
```

Example with `person`:

```
struct person
{
    constexpr static auto serialize(auto & archive, auto & self)
    {
        return archive(self.name, self.age);
    }

    std::string name;
    int age{};
};
```

Or via argument-dependent lookup, leaving the class definition untouched:

```
namespace my_namespace
{
struct person
{
    std::string name;
    int age{};
};

constexpr auto serialize(auto & archive, person & person)
{
    return archive(person.name, person.age);
}

constexpr auto serialize(auto & archive, const person & person)
{
    return archive(person.name, person.age);
}
} // namespace my_namespace
```

10 Archive Creation

Input and output archives can be created together or separately:

```
// Vector of bytes plus input and output archives.
auto [data, in, out] = zpp::bits::data_in_out();

// Just the in/out archives, bound to an existing vector of bytes.
std::vector<std::byte> data;
auto [in, out] = zpp::bits::in_out(data);

// Each archive separately.
std::vector<std::byte> data;
zpp::bits::in in(data);
zpp::bits::out out(data);

// When you only need one direction:
auto [data, in] = zpp::bits::data_in();
auto [data, out] = zpp::bits::data_out();
```

Archives accept any common byte container:

```
// Any of these work as the backing storage.
std::vector<std::byte> data;
std::vector<char> data;
std::vector<unsigned char> data;
std::string data;

zpp::bits::in in(data);
zpp::bits::out out(data);
```

Fixed-size buffers — built-in arrays, `std::array`, and `std::span` — work the same way, but cannot grow, so make sure they are large enough:

```
std::byte data[0x1000];
char data[0x1000];
unsigned char data[0x1000];
std::array<std::byte, 0x1000> data;
std::array<char, 0x1000> data;
std::array<unsigned char, 0x1000> data;
std::span<std::byte> data = /*...*/;
std::span<char> data = /*...*/;
std::span<unsigned char> data = /*...*/;

zpp::bits::in in(data);
zpp::bits::out out(data);
```

`std::vector` and `std::string` grow automatically; fixed-size buffers are strictly capped at their size. All archive constructors accept a variadic list of control options — byte order, default size type, append behaviour, etc. — covered in later sections.

11 Constexpr Serialization

The library is almost entirely `constexpr`. The following uses an array as both the data buffer and the serialization target at compile time:

```
constexpr auto tuple_integers()
{
    std::array<std::byte, 0x1000> data{};
    auto [in, out] = zpp::bits::in_out(data);
    out(std::tuple{1,2,3,4,5}).or_throw();

    std::tuple t{0,0,0,0,0};
    in(t).or_throw();
    return t;
}

// Compile-time check.
static_assert(tuple_integers() == std::tuple{1,2,3,4,5});
```

Simplified compile-time helpers are also provided:

```
using namespace zpp::bits::literals;

// Returns an array containing the bytes of "Hello World!" followed by
// 1337 encoded as a 4-byte integer.
constexpr std::array data =
    zpp::bits::to_bytes<"Hello World!"_s, 1337>();

static_assert(
    zpp::bits::from_bytes<data,
                          zpp::bits::string_literal<char, 12>,
                          int>() == std::tuple{"Hello World!"_s, 1337});
```

12 Position Control

Query the current position of `in` and `out` using `position()` — the number of bytes read or written so far:

```
std::size_t bytes_read    = in.position();
std::size_t bytes_written = out.position();
```

The position can also be moved manually. Use with care:

```
in.reset();                    // reset to beginning.
in.reset(position);            // reset to position.
in.position() -= sizeof(int);  // Go back an integer.
in.position() += sizeof(int);  // Go forward an integer.

out.reset();
out.reset(position);
out.position() -= sizeof(int);
out.position() += sizeof(int);
```

13 Standard Library Types Serialization

For variable-length types like vectors, strings, and view types like `std::span` and `std::string_view`, the library prefixes the data with a 4-byte size (`std::uint32_t`):

```
std::vector v = {1,2,3,4};
out(v);
in(v);
```

The 4-byte default is portable across architectures and matches the practical reality that most programs never need more than 2^{32} elements per container — paying 8 bytes by default would be wasteful.

For other sizes, use `zpp::bits::sized` or `zpp::bits::sized_t`:

```
// Using `sized`:
std::vector<int> v = {1,2,3,4};
out(zpp::bits::sized<std::uint16_t>(v));
in(zpp::bits::sized<std::uint16_t>(v));

// Using `sized_t`:
zpp::bits::sized_t<std::vector<int>, std::uint16_t> v = {1,2,3,4};
out(v);
in(v);
```

Caveat: make sure the size type is large enough — otherwise items are silently dropped according to unsigned conversion rules.

The size can also be omitted entirely:

```
// Using `unsized`:
std::vector<int> v = {1,2,3,4};
out(zpp::bits::unsized(v));
in(zpp::bits::unsized(v));

// Using `unsized_t`:
zpp::bits::unsized_t<std::vector<int>> v = {1,2,3,4};
out(v);
in(v);
```

Alias declarations cover the most common size variants. The same pattern is applied to `string`, `string_view`, `wstring`, the `u8`/`u16`/`u32` flavours, and `span`:

```
zpp::bits::vector1b<T>;      // vector with 1 byte size.
zpp::bits::vector2b<T>;      // vector with 2 byte size.
zpp::bits::vector4b<T>;      // vector with 4 byte size == default std::vector
configuration.
zpp::bits::vector8b<T>;      // vector with 8 byte size.
zpp::bits::static_vector<T>; // unsized vector.
zpp::bits::native_vector<T>; // vector with native (size_type) byte size.

zpp::bits::span1b<T>;        // span with 1 byte size.
zpp::bits::span2b<T>;        // span with 2 byte size.
zpp::bits::span4b<T>;        // span with 4 byte size == default std::span
configuration.
```

```
zpp::bits::span8b<T>;        // span with 8 byte size.
zpp::bits::static_span<T>;   // unsized span.
zpp::bits::native_span<T>;   // span with native (size_type) byte size.
```

Fixed-size types — built-in arrays, `std::array`, `std::tuple` — carry no size prefix; their elements are stored back-to-back.

To change the default size type for an entire archive, pass an option at construction:

```
zpp::bits::in in(data, zpp::bits::size1b{});         // 1 byte
zpp::bits::out out(data, zpp::bits::size1b{});

zpp::bits::in in(data, zpp::bits::size2b{});         // 2 bytes
zpp::bits::out out(data, zpp::bits::size2b{});

zpp::bits::in in(data, zpp::bits::size4b{});         // 4 bytes (default)
zpp::bits::out out(data, zpp::bits::size4b{});

zpp::bits::in in(data, zpp::bits::size8b{});         // 8 bytes
zpp::bits::out out(data, zpp::bits::size8b{});

zpp::bits::in in(data, zpp::bits::size_native{});    // std::size_t
zpp::bits::out out(data, zpp::bits::size_native{});

zpp::bits::in in(data, zpp::bits::no_size{});        // no size prefix — special
cases only
zpp::bits::out out(data, zpp::bits::no_size{});

// Same idea with the convenience factories:
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::size2b{});
auto [data, out]     = zpp::bits::data_out(zpp::bits::size2b{});
auto [data, in]      = zpp::bits::data_in(zpp::bits::size2b{});
```

14 Serialization as Bytes

For most types, the library automatically detects when serialization can be optimised to a raw `memcpy`. This optimisation is disabled when an explicit serialization function is used.

If you know your type is safely byte-serializable and you are using explicit serialization, you can opt back in:

```
struct point
{
    int x;
    int y;

    constexpr static auto serialize(auto & archive, auto & self)
    {
        // Serialize as bytes, instead of member-by-member. The end result is the
        // same, but this is sometimes faster.
        return archive(zpp::bits::as_bytes(self));
    }
};
```

The same idea applies to a vector or span of trivially copyable types. Note that `bytes` (not `as_bytes`) is used here — we are reinterpreting the container's contents, not the container itself:

```
std::vector<point> points;
out(zpp::bits::bytes(points));
in(zpp::bits::bytes(points));
```

In this form the size is **not** serialized. As a workaround, cast to `std::span<std::byte>` if a size prefix is required.

15 Variant Types and Version Control

There is no perfect tool for backwards compatibility in a zero-overhead format, but `std::variant` is a natural fit for versioning and for polymorphic dispatch. Note that under C++26 the explicit `using serialize = members<N>;` declarations below are **not** required — they are shown only because the difference in member count is what distinguishes the two versions semantically:

```
namespace v1
{
struct person
{
    auto get_hobby() const
    {
        return "<none>"sv;
    }

    std::string name;
    int age;
};
} // namespace v1

namespace v2
{
struct person
{
    auto get_hobby() const
    {
        return std::string_view(hobby);
    }

    std::string name;
    int age;
    std::string hobby;
};
} // namespace v2
```

Serializing through the variant:

```
auto [data, in, out] = zpp::bits::data_in_out();
out(std::variant<v1::person, v2::person>(v1::person{"Person1", 25}))
    .or_throw();

std::variant<v1::person, v2::person> v;
in(v).or_throw();

std::visit([](auto && person) {
    (void) person.name == "Person1";
    (void) person.age == 25;
    (void) person.get_hobby() == "<none>";
}, v);

out(std::variant<v1::person, v2::person>(
        v2::person{"Person2", 35, "Basketball"}))
    .or_throw();
```

```
in(v).or_throw();

std::visit([](auto && person) {
    (void) person.name == "Person2";
    (void) person.age == 35;
    (void) person.get_hobby() == "Basketball";
}, v);
```

By default the variant's index (0 or 1) is serialized as a single `std::byte` ahead of the payload. This is very efficient but sometimes an explicit serialization ID is preferable — see below.

15.1 Custom serialization IDs

Add one of the following inside or outside the class:

```
using namespace zpp::bits::literals;

// Inside: serializes the full string "v1::person" before the person.
using serialize_id = zpp::bits::id<"v1::person"_s>;

// Outside: same effect via ADL.
auto serialize_id(const person &) -> zpp::bits::id<"v1::person"_s>;
```

All serialization IDs within a single variant must match in length, or compilation fails.

Any byte sequence works as an ID — strings, integers, hashes, or any literal type. For instance, using a hash:

```
using namespace zpp::bits::literals;

// Inside:
using serialize_id = zpp::bits::id<"v1::person"_sha1>;     // SHA-1
using serialize_id = zpp::bits::id<"v1::person"_sha256>;   // SHA-256

// Outside:
auto serialize_id(const person &) -> zpp::bits::id<"v1::person"_sha1>;
auto serialize_id(const person &) -> zpp::bits::id<"v1::person"_sha256>;
```

Use only the first N bytes of the hash if needed:

```
// First 4 bytes:
using serialize_id = zpp::bits::id<"v1::person"_sha256, 4>;

// First sizeof(int) bytes:
using serialize_id = zpp::bits::id<"v1::person"_sha256_int>;
```

Internally, the ID type is converted to bytes at compile time using `zpp::bits::out`. Any literal type that can be serialized is eligible. The result is stored as `std::array<std::byte, N>`; for the special sizes 1, 2, 4, and 8 the underlying type is `std::byte`, `std::uint16_t`, `std::uint32_t`, or `std::uint64_t` respectively for ease of use.

15.2 Known IDs

To skip serializing the ID — when the receiver already knows the variant alternative — wrap the variant with `zpp::bits::known_id`:

```
std::variant<v1::person, v2::person> v;

// Id assumed to be v2::person; not serialized / deserialized.
out(zpp::bits::known_id<"v2::person"_sha256_int>(v));
in(zpp::bits::known_id<"v2::person"_sha256_int>(v));

// You can also pass the id as a runtime parameter. `id_v` ("id value") gives
// you the literal as a regular integer where applicable; here 4 bytes maps to
// std::uint32_t.
in(zpp::bits::known_id(zpp::bits::id_v<"v2::person"_sha256_int>, v));
```

16 Literal Operators

Helper user-defined literals provided by the library (in the `literals` inline namespace):

```
using namespace zpp::bits::literals;

"hello"_s             // String literal as `string_literal`.
"hello"_b             // Raw binary data literal.
"hello"_sha1          // SHA-1 of "hello" as a binary literal.
"hello"_sha256        // SHA-256 of "hello" as a binary literal.
"hello"_sha1_int      // SHA-1 truncated to int width.
"hello"_sha256_int    // SHA-256 truncated to int width.
"01020304"_decode_hex // Decode a hex string into bytes.
```

17 Apply to Functions

The contents of an input archive can be applied directly to a function call with `zpp::bits::apply`. The function must be non-template and have exactly one overload:

```
int foo(std::string s, int i)
{
    // s == "hello"s;
    // i == 1337;
    return 1338;
}

auto [data, in, out] = zpp::bits::data_in_out();
out("hello"s, 1337).or_throw();

// Exception based:
zpp::bits::apply(foo, in).or_throw() == 1338;

// zpp::throwing based:
co_await zpp::bits::apply(foo, in) == 1338;

// Return value based:
if (auto result = zpp::bits::apply(foo, in);
    failure(result)) {
    // Failure...
} else {
    result.value() == 1338;
}
```

If the function takes no parameters, the call is invoked without deserialization and the return value is that of the function. If the function returns void, no value is produced.

18 Remote Procedure Call (RPC)

The module provides a thin RPC interface for serializing and deserializing function calls:

```
using namespace std::literals;
using namespace zpp::bits::literals;

int foo(int i, std::string s);
std::string bar(int i, int j);

using rpc = zpp::bits::rpc<
    zpp::bits::bind<foo, "foo"_sha256_int>,
    zpp::bits::bind<bar, "bar"_sha256_int>
>;

auto [data, in, out] = zpp::bits::data_in_out();

// Server and client together:
auto [client, server] = rpc::client_server(in, out);

// Or separately:
rpc::client client{in, out};
rpc::server server{in, out};

// Request from the client:
client.request<"foo"_sha256_int>(1337, "hello"s).or_throw();

// Serve the request from the server:
server.serve().or_throw();

// Read back the response:
client.response<"foo"_sha256_int>().or_throw(); // == foo(1337, "hello"s)
```

RPC supports all three error-handling styles:

```
// Return-value based:
if (auto result = client.request<"foo"_sha256_int>(1337, "hello"s);
failure(result)) {
    // handle
}
if (auto result = server.serve(); failure(result)) {
    // handle
}
if (auto result = client.response<"foo"_sha256_int>(); failure(result)) {
    // handle
} else {
    // result.value()
}

// Throwing based:
co_await client.request<"foo"_sha256_int>(1337, "hello"s);
co_await server.serve();
co_await client.response<"foo"_sha256_int>(); // == foo(1337, "hello"s)
```

If the call ID is communicated out of band, it can be skipped:

```
server.serve(id);                       // id is already known
client.request_body<Id>(arguments...);  // request without serializing id
```

18.1 Member functions

Member functions can be bound too. The server must receive a reference to the object, and all member functions must belong to the same class — though they can freely be mixed with namespace-scope functions:

```
struct a
{
    int foo(int i, std::string s);
};

std::string bar(int i, int j);

using rpc = zpp::bits::rpc<
    zpp::bits::bind<&a::foo, "a::foo"_sha256_int>,
    zpp::bits::bind<bar, "bar"_sha256_int>
>;

auto [data, in, out] = zpp::bits::data_in_out();

a a1;

// Server and client together:
auto [client, server] = rpc::client_server(in, out, a1);

// Or separately:
rpc::client client{in, out};
rpc::server server{in, out, a1};

client.request<"a::foo"_sha256_int>(1337, "hello"s).or_throw();
server.serve().or_throw();
client.response<"a::foo"_sha256_int>().or_throw(); // == a1.foo(1337, "hello"s)
```

18.2 Opaque mode

When a function manages its own serialization, bind it with `bind_opaque`. Any of these signatures are valid:

```
auto foo(zpp::bits::in<> &, zpp::bits::out<> &);
auto foo(zpp::bits::in<> &);
auto foo(zpp::bits::out<> &);
auto foo(std::span<std::byte> input);   // assumes all data consumed from archive.
auto foo(std::span<std::byte> & input); // resize to signal how much was consumed.

using rpc = zpp::bits::rpc<
    zpp::bits::bind_opaque<foo, "a::foo"_sha256_int>,
    zpp::bits::bind<bar, "bar"_sha256_int>
>;
```

19 Byte Order Customization

The default byte order is the platform's native one. To force a specific order, pass `zpp::bits::endian::*` at construction:

```
zpp::bits::in in(data, zpp::bits::endian::big{});      // Use big endian
zpp::bits::out out(data, zpp::bits::endian::big{});

zpp::bits::in in(data, zpp::bits::endian::network{});  // Alias for big endian
zpp::bits::out out(data, zpp::bits::endian::network{});

zpp::bits::in in(data, zpp::bits::endian::little{});   // Use little endian
zpp::bits::out out(data, zpp::bits::endian::little{});

zpp::bits::in in(data, zpp::bits::endian::swapped{});  // Opposite of native
zpp::bits::out out(data, zpp::bits::endian::swapped{});

zpp::bits::in in(data, zpp::bits::endian::native{});   // Default
zpp::bits::out out(data, zpp::bits::endian::native{});

// Same idea with the convenience factories:
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::endian::big{});
auto [data, out]     = zpp::bits::data_out(zpp::bits::endian::big{});
auto [data, in]      = zpp::bits::data_in(zpp::bits::endian::big{});
```

20 Deserializing Views Of Const Bytes

On the input side, the library supports view types over const bytes — for example `std::span<const std::byte>` — so a section of the buffer can be aliased without copying. Handle with care: invalidating the underlying storage produces a use-after-free.

```
using namespace std::literals;

auto [data, in, out] = zpp::bits::data_in_out();
out("hello"sv).or_throw();

std::span<const std::byte> s;
in(s).or_throw();

// s.size() == "hello"sv.size()
// std::memcmp("hello"sv.data(), s.data(), "hello"sv.size()) == 0
```

An unsized variant consumes the rest of the archive, useful for header-plus-payload protocols:

```
auto [data, in, out] = zpp::bits::data_in_out();
out(zpp::bits::unsized("hello"sv)).or_throw();

std::span<const std::byte> s;
in(zpp::bits::unsized(s)).or_throw();

// s.size() == "hello"sv.size()
// std::memcmp("hello"sv.data(), s.data(), "hello"sv.size()) == 0
```

21 Pointers as Optionals

The library does not serialize null pointers by default. To represent an optional owning pointer (useful for graphs and other recursive structures), use `zpp::bits::optional_ptr<T>` instead of `std::optional<std::unique_ptr<T>>`.

`optional_ptr<T>` is more efficient: it omits the boolean discriminator that `std::optional` carries, using the null pointer itself as the empty state. The wire format matches `std::optional<T>` — a zero byte for null, or a one byte followed by the serialized value.

22 Reflection

Several reflection primitives that the module uses internally are exposed for user code. Under C++26 these work on plain aggregates without any extra declaration:

```
struct point
{
    int x;
    int y;
};

static_assert(zpp::bits::number_of_members<point>() == 2);

constexpr auto sum = zpp::bits::visit_members(
    point{1, 2}, [](auto x, auto y) { return x + y; });

static_assert(sum == 3);

constexpr auto generic_sum = zpp::bits::visit_members(
    point{1, 2}, [](auto... members) { return (0 + ... + members); });

static_assert(generic_sum == 3);

constexpr auto is_two_integers =
    zpp::bits::visit_members_types<point>([]<typename... Types>() {
        if constexpr (std::same_as<std::tuple<Types...>,
                                   std::tuple<int, int>>) {
            return std::true_type{};
        } else {
            return std::false_type{};
        }
    })();

static_assert(is_two_integers);
```

23 Additional Archive Controls

Archives accept a variety of control options at construction. `append{}` positions an output archive at the end of an existing buffer (no effect on input archives):

```
std::vector<std::byte> data;
zpp::bits::out out(data, zpp::bits::append{});
```

Multiple controls compose, and they apply equally to `data_in_out`/`data_in`/`data_out`/`in_out`:

```
zpp::bits::out out(data, zpp::bits::append{}, zpp::bits::endian::big{});
auto [in, out]       = zpp::bits::in_out(data, zpp::bits::append{},
zpp::bits::endian::big{});
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::size2b{},
zpp::bits::endian::big{});
```

23.1 Allocation limits

`zpp::bits::alloc_limit<L>` caps allocations — useful as a sanity check against oversized length-prefixed messages, not as an exact accounting mechanism:

```
zpp::bits::out out(data, zpp::bits::alloc_limit<0x10000>{});
zpp::bits::in in(data, zpp::bits::alloc_limit<0x10000>{});
auto [in, out]       = zpp::bits::in_out(data, zpp::bits::alloc_limit<0x10000>{});
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::alloc_limit<0x10000>{});
```

23.2 Avoiding final resize

By default a growing output buffer is resized at the end to match the final position. To skip this final resize and detect the end via `position()` instead, use `no_fit_size{}`:

```
zpp::bits::out out(data, zpp::bits::no_fit_size{});
```

23.3 Enlargement strategy

Control how the output buffer grows with `zpp::bits::enlarger<Mul, Div = 1>`:

```
zpp::bits::out out(data, zpp::bits::enlarger<2>{});      // 2x on grow.
zpp::bits::out out(data, zpp::bits::enlarger<3, 2>{});   // Default — 1.5x.
zpp::bits::out out(data, zpp::bits::exact_enlarger{});   // Grow to exact size.
```

23.4 Disabling overflow checks

By default, a growing output buffer checks for size overflow before each enlargement. On 64-bit systems this is essentially free but redundant — allocations fail long before a 64-bit size overflows. To skip the check:

```
zpp::bits::out out(data, zpp::bits::no_enlarge_overflow{});
```

23.5 Detecting archive kind in explicit serialize

When writing an explicit `serialize` function you often need to branch on whether the archive is input or output. Use `archive.kind()` inside an `if constexpr`:

```
static constexpr auto serialize(auto & archive, auto & self)
{
    using archive_type = std::remove_cvref_t<decltype(archive)>;

    if constexpr (archive_type::kind() == zpp::bits::kind::in) {
        // Input archive
    } else if constexpr (archive_type::kind() == zpp::bits::kind::out) {
        // Output archive
    }
}
```

24 Variable Length Integers

The module provides variable-length integer types:

```
auto [data, in, out] = zpp::bits::data_in_out();
out(zpp::bits::varint{150}).or_throw();

zpp::bits::varint i{0};
in(i).or_throw();

// i == 150
```

The encoding can be verified at compile time:

```
static_assert(zpp::bits::to_bytes<zpp::bits::varint{150}>() == "9601"_decode_hex);
```

The class template `zpp::bits::varint<T, E = varint_encoding::normal>` accepts any integer or enumeration type, with `varint_encoding::normal` or `varint_encoding::zig_zag` as the encoding. The following aliases are predefined:

```
using vint32_t  = varint<std::int32_t>;    // int32 varint
using vint64_t  = varint<std::int64_t>;    // int64 varint

using vuint32_t = varint<std::uint32_t>;   // unsigned int32 varint
using vuint64_t = varint<std::uint64_t>;   // unsigned int64 varint

using vsint32_t = varint<std::int32_t, varint_encoding::zig_zag>; // zig-zag int32
using vsint64_t = varint<std::int64_t, varint_encoding::zig_zag>; // zig-zag int64

using vsize_t   = varint<std::size_t>;     // std::size_t varint
```

Varints can also serve as the default size type for an archive:

```
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::size_varint{});

zpp::bits::in  in(data,  zpp::bits::size_varint{}); // Uses varint to encode size.
zpp::bits::out out(data, zpp::bits::size_varint{});
```

25 Protobuf

The library's native format is not based on any pre-existing wire format, which makes cross-language communication impractical. To bridge that gap, the library also supports the Protobuf wire format.

Note: Protobuf support is experimental. Not every Protobuf feature is implemented, and it is roughly 2–5× slower than the default format (mostly on deserialization), which is designed to be zero-overhead.

A minimal message:

```
struct example
{
    zpp::bits::vint32_t i; // varint of 32 bit, field number is implicitly 1,
                           // next field is implicitly 2, and so on.
};

// Serialize as protobuf protocol. (As usual, can also go inside the class
// with `using serialize = zpp::bits::pb_protocol;`.)
auto serialize(const example &) -> zpp::bits::pb_protocol;

// Use archives as usual. We pick no_size to expose the raw encoding, but in
// production protobuf messages should normally be size-prefixed since they are
// not self-terminating.
auto [data, in, out] = zpp::bits::data_in_out(zpp::bits::no_size{});

out(example{.i = 150}).or_throw();

example e;
in(e).or_throw();

// e.i == 150

// Compile-time check of the encoding:
static_assert(
    zpp::bits::to_bytes<zpp::bits::unsized_t<example>{{.i = 150}}>() ==
    "089601"_decode_hex);
```

For the full syntax (used later to pass options), use `zpp::bits::protocol`:

```
auto serialize(const example &) -> zpp::bits::protocol<zpp::bits::pb{}>;
```

25.1 Field numbering

Reserve fields with `pb_reserved`:

```
struct example
{
    [[no_unique_address]] zpp::bits::pb_reserved _1; // field number 1 reserved
    zpp::bits::vint32_t i;  // field number == 2
    zpp::bits::vsint32_t j; // field number == 3
};
```

Specify field numbers explicitly with `pb_field`:

```
struct example
{
    zpp::bits::pb_field<zpp::bits::vint32_t, 20> i;  // field number == 20
    zpp::bits::pb_field<zpp::bits::vsint32_t, 30> j; // field number == 30

    using serialize = zpp::bits::pb_protocol;
};
```

Access to a `pb_field` is normally transparent. When explicit access is needed, use `pb_value(variable)` to get or set the underlying value.

Remap member positions to different field numbers with `pb_map`:

```
struct example
{
    zpp::bits::vint32_t i;  // field number == 20
    zpp::bits::vsint32_t j; // field number == 30

    using serialize = zpp::bits::protocol<
        zpp::bits::pb{
            zpp::bits::pb_map<1, 20>{},  // first member -> field 20
            zpp::bits::pb_map<2, 30>{}}>; // second member -> field 30
};
```

Fixed members are plain C++ data members:

```
struct example
{
    std::uint32_t i; // fixed unsigned integer 32, field number == 1
};
```

If a member count is required (it usually is not under C++26), use `pb_members<N>`:

```
struct example
{
    using serialize = zpp::bits::pb_members<1>;

    zpp::bits::vint32_t i; // field number == 1
};
```

The fully spelled-out form passes the count as the second parameter to `protocol`:

```
struct example
{
    using serialize = zpp::bits::protocol<zpp::bits::pb{}, 1>;

    zpp::bits::vint32_t i; // field number == 1
};
```

25.2 Embedded and repeated fields

Embedded messages are nested data members:

```cpp
struct nested_example
{
    example nested; // field number == 1
};

auto serialize(const nested_example &) -> zpp::bits::pb_protocol;

static_assert(zpp::bits::to_bytes<zpp::bits::unsized_t<nested_example>{
                  {.nested = example{150}}}>() == "0a03089601"_decode_hex);
```

Repeated fields are owning containers:

```cpp
struct repeating
{
    using serialize = zpp::bits::pb_protocol;

    std::vector<zpp::bits::vint32_t> integers; // field number == 1
    std::string characters;                    // field number == 2
    std::vector<example> examples;             // field number == 3
};
```

All fields are currently optional, which is good practice — missing fields are dropped rather than concatenated, which is more efficient. A value not present in the wire data leaves the target member intact, so defaults can be set via non-static data member initializers or by initializing the destination before deserializing.

25.3 Full example: translating a .proto file

Original Protobuf:

```
syntax = "proto3";

package tutorial;

message person {
  string name = 1;
  int32 id = 2;
  string email = 3;

  enum phone_type {
    mobile = 0;
    home = 1;
    work = 2;
  }

  message phone_number {
    string number = 1;
    phone_type type = 2;
  }

  repeated phone_number phones = 4;
}

message address_book {
```

```
  repeated person people = 1;
}
```

Translated to C++:

```
import zpp.bits;

struct person
{
    std::string name;       // = 1
    zpp::bits::vint32_t id; // = 2
    std::string email;      // = 3

    enum phone_type
    {
        mobile = 0,
        home   = 1,
        work   = 2,
    };

    struct phone_number
    {
        std::string number; // = 1
        phone_type type;    // = 2
    };

    std::vector<phone_number> phones; // = 4
};

struct address_book
{
    std::vector<person> people; // = 1
};

auto serialize(const person &) -> zpp::bits::pb_protocol;
auto serialize(const person::phone_number &) -> zpp::bits::pb_protocol;
auto serialize(const address_book &) -> zpp::bits::pb_protocol;
```

Suppose a message was serialized from Python:

```
import addressbook_pb2
person = addressbook_pb2.person()
person.id = 1234
person.name = "John Doe"
person.email = "jdoe@example.com"
phone = person.phones.add()
phone.number = "555-4321"
phone.type = addressbook_pb2.person.home
```

Inspecting it:

```
name: "John Doe"
id: 1234
email: "jdoe@example.com"
```

```
phones {
  number: "555-4321"
  type: home
}
```

Serializing it:

```
person.SerializeToString()
```

The result:

```
b'\n\x08John Doe\x10\xd2\t\x1a\x10jdoe@example.com"\x0c\n\x08555-4321\x10\x01'
```

Reading it back in C++:

```
using namespace zpp::bits::literals;

constexpr auto data =
    "\n\x08John Doe\x10\xd2\t\x1a\x10jdoe@example.com\"\x0c\n\x08"
    "555-4321\x10\x01"_b;
static_assert(data.size() == 45);

person p;
zpp::bits::in{data, zpp::bits::no_size{}}(p).or_throw();

// p.name == "John Doe"
// p.id == 1234
// p.email == "jdoe@example.com"
// p.phones.size() == 1
// p.phones[0].number == "555-4321"
// p.phones[0].type == person::home
```

26 Advanced Controls

The inlining knobs from the original header library survive in the port, but they are baked into the BMI at module-build time (see Section 3). They cannot be changed per importer.

On some compilers, recursive structures (for example a tree graph) cause always-inline to fail. The workaround is to provide an explicit `serialize` function for the offending type — the library usually detects such cases automatically, but the example is here in case it does not:

```
struct node
{
    constexpr static auto serialize(auto & archive, auto & node)
    {
        return archive(node.value, node.nodes);
    }

    int value;
    std::vector<node> nodes;
};
```

27 Benchmark

These numbers come from the original `zpp::bits` README and reflect the header library, not this port specifically. The on-wire format and the algorithmic core are unchanged, so the relative ranking should carry over; absolute numbers may differ on your toolchain. Results from `fraillt/cpp_serializers_benchmark`.

27.1 GCC 11

library	test case	bin size	data size	ser time	des time
zpp_bits	general	52192B	8413B	**733ms**	**693ms**
zpp_bits	fixed buffer	48000B	8413B	**620ms**	**667ms**
bitsery	general	70904B	6913B	1470ms	1524ms
bitsery	fixed buffer	53648B	6913B	927ms	1466ms
boost	general	279024B	11037B	15126ms	12724ms
cereal	general	70560B	10413B	10777ms	9088ms
flatbuffers	general	70640B	14924B	8757ms	3361ms
handwritten	general	47936B	10413B	1506ms	1577ms
handwritten	unsafe	47944B	10413B	1616ms	1392ms
iostream	general	53872B	8413B	11956ms	12928ms
msgpack	general	89144B	8857B	2770ms	14033ms
protobuf	general	2077864B	10018B	19929ms	20592ms
protobuf	arena	2077872B	10018B	10319ms	11787ms
yas	general	61072B	10463B	2286ms	1770ms

27.2 Clang 12.0.1

library	test case	bin size	data size	ser time	des time
zpp_bits	general	47128B	8413B	**790ms**	**715ms**
zpp_bits	fixed buffer	43056B	8413B	**605ms**	**694ms**
bitsery	general	53728B	6913B	2128ms	1832ms
bitsery	fixed buffer	49248B	6913B	946ms	1941ms
boost	general	237008B	11037B	16011ms	13017ms
cereal	general	61480B	10413B	9977ms	8565ms
flatbuffers	general	62512B	14924B	9812ms	3472ms
handwritten	general	43112B	10413B	1391ms	1321ms
handwritten	unsafe	43120B	10413B	1393ms	1212ms
iostream	general	48632B	8413B	10992ms	12771ms
msgpack	general	77384B	8857B	3563ms	14705ms
protobuf	general	2032712B	10018B	18125ms	20211ms
protobuf	arena	2032760B	10018B	9166ms	11378ms
yas	general	51000B	10463B	2114ms	1558ms

28 Limitations

- Serialization of non-owning and raw pointers is not supported, for simplicity and security.
- Serialization of null pointers is not supported, to avoid the per-pointer overhead of a presence flag. Use `std::optional` or `zpp::bits::optional_ptr<T>` for explicit nullability.
- The build-time inlining knobs (`ZPP_BITS_INLINE_DECODE_VARINT`, `ZPP_BITS_INLINE_MODE`) cannot be changed by importers; they are fixed at module precompilation time.